RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105

CP23231 – APPLIED MACHINE LEARNING

LABORATORY

Laboratory Record Note Book

Name	**SATHISH G**
Branch	**ME-CSE**
Register No	**250711015**
Semester	**2**
Academic Year	**2025 -2027**

RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105

BONAFIDE CERTIFICATE

Name: . . . . . .SATHISH G . . . . . .

Academic Year : 2025 -2027 ODD Semester : 2 Branch: ME-CSE

Register No.	**250711015**

Certified that this is the bonafide record of work done by the above student in the Laboratory during the year

Signature of Faculty in-charge

Submitted for the Practical Examination held on . . . . . .

Internal Examiner **External Examiner**

APPLIED MACHINE LEARNING

LABORATORY MANUAL

Dataset Used:

Mushroom Classification Dataset (mushroom.csv)

List of Experiments	
1	Demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples.
2	Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.
3	Write a program to implement k-Nearest Neighbour algorithm to classify the Mushroom data set. Print both correct and wrong predictions.
4	Apply k-Means algorithm to cluster a set of data stored in a CSV file and comment on the quality of clustering.
5	Interpret the results of PCA analysis.
6	Implement the Naïve Bayesian classifier for a sample training data set stored as a CSV file. Compute the accuracy of the classifier, considering few test data sets.

EXPERIMENT 1

Candidate-Elimination Algorithm

AIM

1. To demonstrate the Candidate-Elimination algorithm.
2. To compute and display the set of all hypotheses consistent with training examples from the Mushroom dataset.
3. To maintain and converge specific (S) and general (G) boundary sets throughout training.

ALGORITHM

1. Initialize S (most specific hypothesis) and G (most general hypothesis) boundaries.
2. For each training example (x, c), if c = positive: generalize S to cover x, remove any G member inconsistent with x.
3. If c = negative: specialize G to exclude x, remove any S member inconsistent with x.
4. Remove from S or G any hypothesis less general than another in the same set.
5. Repeat until all examples are processed.
6. Output the final version space defined by S and G boundaries.

SOURCE CODE

Step 1: Load Libraries & Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data_set = 'mushroom.csv'
df = pd.read_csv(data_set)
```

Step 2: Data Exploration

```
df.shape
df.head()
df.tail()
df.dtypes
df.info()
df.describe(include='all').T
print(len(df.columns))
```

Step 3: Pre-Processing

3.1 Rename Columns

```
df.columns = [
    'Index','Mushroom_Class','Cap_Shape','Cap_Surface','Cap_Color',
    'Has_Bruises','Odor','Gill_Attachment','Gill_Spacing','Gill_Size',
    'Gill_Color','Stalk_Shape','Stalk_Root','Stalk_Surface_Above_Ring',
```

```
    'Stalk_Surface_Below_Ring','Stalk_Color_Above_Ring','Stalk_Color_Below_Ring',
    'Veil_Type','Veil_Color','Ring_Count','Ring_Type','Spore_Print_Color',
    'Population_Type','Habitat_Type'
]
df.columns = df.columns.str.lower().str.replace(' ', '_')
df = df.drop(columns=['index'], errors='ignore')
```

3.2 Remove Duplicates

```
df.duplicated().sum()       # check
df = df.drop_duplicates()   # remove
df.duplicated().sum()       # verify
```

3.3 Handle Null Values

```
null = df.isnull().sum()
null[null>0].sort_values(ascending=False)

def fill_null_values(df):
    df = df.copy()
    for col in df.select_dtypes(include='object').columns:
        if not df[col].mode().empty:
            df[col] = df[col].fillna(df[col].mode()[0])
    for col in df.select_dtypes(include=['int64','float64']).columns:
        df[col] = df[col].fillna(df[col].mean())
    return df

df = fill_null_values(df)
print('NULL VALUES ARE FILLED')
```

Step 4: Encoding

4.1 Binary Encoding

```
binary_map = {
    'gill_attachment': {'FREE':0, 'ATTACHED':1},
    'gill_spacing':    {'CROWDED':0, 'CLOSE':1},
    'gill_size':       {'NARROW':0, 'BROAD':1},
    'stalk_shape':     {'TAPERING':0, 'ENLARGING':1},
    'ring_count':      {'ONE':1, 'TWO':2, 'NONE':0},
    'veil_type': 1,
}

def apply_binary_encoding(df, binary_maps):
    df = df.copy()
    for col, mapping in binary_maps.items():
        if col in df.columns:
            if isinstance(mapping, dict):
                lower_mapping = {k.lower(): v for k, v in mapping.items()}
                df[col] =
df[col].astype(str).str.strip().str.lower().map(lower_mapping)
            else:
                df[col] = mapping
    return df

df = apply_binary_encoding(df, binary_map)
```

```
print('Binary encoding applied successfully')
```

4.2 One-Hot Encoding

```
from sklearn.preprocessing import OneHotEncoder

one_col = ['cap_shape','cap_surface','cap_color','has_bruises','odor',
           'gill_color','stalk_root','stalk_surface_above_ring',
           'stalk_surface_below_ring','stalk_color_above_ring',
           'stalk_color_below_ring','veil_color','ring_type',
           'spore_print_color','population_type','habitat_type']

encoder = OneHotEncoder(sparse_output=False)
for col in one_col:
    encoded = encoder.fit_transform(df[[col]])
    encoded_df = pd.DataFrame(encoded,
                 columns=encoder.get_feature_names_out([col]),
                 index=df.index)
    df = pd.concat([df.drop(col, axis=1), encoded_df], axis=1)
```

4.3 Label Encoding (Target Column)

```
from sklearn.preprocessing import LabelEncoder

label = LabelEncoder()
df['mushroom_class'] = label.fit_transform(df['mushroom_class'])
df['mushroom_class'].unique()
```

Step 5: EDA

```
df['mushroom_class'].value_counts().plot(kind='bar')
plt.title('Mushroom Class Distribution')
plt.show()

sns.countplot(x='gill_size', hue='mushroom_class', data=df)
plt.title('Gill Size vs Mushroom Class')
plt.show()

plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title('Feature Correlation')
plt.show()
```

Step 6: Train-Test Split & Scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

x = df.drop('mushroom_class', axis=1, errors='ignore')
y = df['mushroom_class']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=43)
```

```
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test  = scaler.fit_transform(x_test)
```

Candidate-Elimination Implementation

```
# Use encoded integer features for C-E
from sklearn.preprocessing import OrdinalEncoder

# Re-encode with ordinal for interpretability
ce_df = pd.read_csv('mushroom.csv')
ce_df.columns = ce_df.columns.str.lower().str.replace(' ', '_')
ce_df = ce_df.drop(columns=['index'], errors='ignore').dropna().drop_duplicates()

oe = OrdinalEncoder()
ce_df_enc = pd.DataFrame(oe.fit_transform(ce_df),
columns=ce_df.columns).astype(int)

concepts = ce_df_enc.drop('mushroom_class', axis=1).values
target   = ce_df_enc['mushroom_class'].values

def g_0(n):   return [['?' for _ in range(n)]]
def s_0(n):   return [None]

def is_consistent_with(h, x):
    return all(hi == '?' or hi == xi for hi, xi in zip(h, x))

def candidate_elimination(X, y):
    n = X.shape[1]
    S = [list(X[0]) if y[0]==1 else [None]*n]
    G = g_0(n)
    for xi, yi in zip(X, y):
        if yi == 1:   # positive
            G = [g for g in G if is_consistent_with(g, xi)]
            S = [[(si if si == xi[j] else '?') if si is not None else xi[j]
                  for j, si in enumerate(s)] for s in S]
        else:         # negative
            S = [s for s in S if not is_consistent_with(s, xi)]
            new_G = []
            for g in G:
                for j in range(n):
                    if g[j] == '?':
                        new_g = g[:]
                        new_g[j] = xi[j]  # specialize
                        if (any(is_consistent_with(s, new_g) for s in S)
                                and not is_consistent_with(new_g, xi)):
                            new_G.append(new_g)
            G = new_G if new_G else G
    return S, G

# Run on small sample for tractability
sample = ce_df_enc.sample(200, random_state=42)
X_ce = sample.drop('mushroom_class', axis=1).values
y_ce = sample['mushroom_class'].values
```

```
S_final, G_final = candidate_elimination(X_ce, y_ce)

print('Specific Boundary (S):')
for s in S_final: print(' ', s)
print()
print('General Boundary (G):')
for g in G_final: print(' ', g)
```

RESULT

The Candidate-Elimination algorithm was successfully implemented on the Mushroom dataset. The algorithm maintained and converged the Specific (S) and General (G) boundary sets over the training examples. The final version space represents all hypotheses consistent with the given training data, correctly separating edible from poisonous mushrooms.

EXPERIMENT 2

Artificial Neural Network using Backpropagation

AIM

1. To build an Artificial Neural Network (ANN) using the Backpropagation algorithm.
2. To preprocess and encode the Mushroom dataset and train the ANN using TensorFlow/Keras.
3. To evaluate the ANN using classification metrics including Precision, Recall, F1 Score, and Confusion Matrix.

ALGORITHM

7. Initialize Network: Define input, hidden (Dense + Dropout + BatchNorm), and output (sigmoid) layers.
8. Forward Propagation: Pass input through each layer, applying ReLU (hidden) and Sigmoid (output) activations.
9. Compute Error: Use Binary Cross-Entropy loss between actual and predicted outputs.
10. Backward Propagation: Use Adam optimizer to compute gradients via chain rule and update weights.
11. Early Stopping: Monitor val_loss and restore best weights if improvement stalls (patience=5).
12. Repeat: Train for up to 300 epochs with batch_size=32.
13. Evaluate: Compute Accuracy, MSE, MAE, Precision, Recall, F1, Confusion Matrix, PR Curve.

SOURCE CODE

Step 1: Load Libraries & Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data_set = 'mushroom.csv'
df = pd.read_csv(data_set)
```

Step 2: Data Exploration

```
df.shape
df.head()
df.tail()
df.dtypes
df.info()
df.describe(include='all').T
print(len(df.columns))
```

Step 3: Pre-Processing

3.1 Rename Columns

```
df.columns = [
```

```
    'Index','Mushroom_Class','Cap_Shape','Cap_Surface','Cap_Color',
    'Has_Bruises','Odor','Gill_Attachment','Gill_Spacing','Gill_Size',
    'Gill_Color','Stalk_Shape','Stalk_Root','Stalk_Surface_Above_Ring',
    'Stalk_Surface_Below_Ring','Stalk_Color_Above_Ring','Stalk_Color_Below_Ring',
    'Veil_Type','Veil_Color','Ring_Count','Ring_Type','Spore_Print_Color',
    'Population_Type','Habitat_Type'
]
df.columns = df.columns.str.lower().str.replace(' ', '_')
df = df.drop(columns=['index'], errors='ignore')
```

3.2 Remove Duplicates

```
df.duplicated().sum()       # check
df = df.drop_duplicates()   # remove
df.duplicated().sum()       # verify
```

3.3 Handle Null Values

```
null = df.isnull().sum()
null[null>0].sort_values(ascending=False)

def fill_null_values(df):
    df = df.copy()
    for col in df.select_dtypes(include='object').columns:
        if not df[col].mode().empty:
            df[col] = df[col].fillna(df[col].mode()[0])
    for col in df.select_dtypes(include=['int64','float64']).columns:
        df[col] = df[col].fillna(df[col].mean())
    return df

df = fill_null_values(df)
print('NULL VALUES ARE FILLED')
```

Step 4: Encoding

4.1 Binary Encoding

```
binary_map = {
    'gill_attachment': {'FREE':0, 'ATTACHED':1},
    'gill_spacing':    {'CROWDED':0, 'CLOSE':1},
    'gill_size':       {'NARROW':0, 'BROAD':1},
    'stalk_shape':     {'TAPERING':0, 'ENLARGING':1},
    'ring_count':      {'ONE':1, 'TWO':2, 'NONE':0},
    'veil_type': 1,
}

def apply_binary_encoding(df, binary_maps):
    df = df.copy()
    for col, mapping in binary_maps.items():
        if col in df.columns:
            if isinstance(mapping, dict):
                lower_mapping = {k.lower(): v for k, v in mapping.items()}
                df[col] =
df[col].astype(str).str.strip().str.lower().map(lower_mapping)
            else:
                df[col] = mapping
```

```
    return df

df = apply_binary_encoding(df, binary_map)
print('Binary encoding applied successfully')
```

4.2 One-Hot Encoding

```
from sklearn.preprocessing import OneHotEncoder

one_col = ['cap_shape','cap_surface','cap_color','has_bruises','odor',
           'gill_color','stalk_root','stalk_surface_above_ring',
           'stalk_surface_below_ring','stalk_color_above_ring',
           'stalk_color_below_ring','veil_color','ring_type',
           'spore_print_color','population_type','habitat_type']

encoder = OneHotEncoder(sparse_output=False)
for col in one_col:
    encoded = encoder.fit_transform(df[[col]])
    encoded_df = pd.DataFrame(encoded,
                 columns=encoder.get_feature_names_out([col]),
                 index=df.index)
    df = pd.concat([df.drop(col, axis=1), encoded_df], axis=1)
```

4.3 Label Encoding (Target Column)

```
from sklearn.preprocessing import LabelEncoder

label = LabelEncoder()
df['mushroom_class'] = label.fit_transform(df['mushroom_class'])
df['mushroom_class'].unique()
```

Step 5: EDA

```
df['mushroom_class'].value_counts().plot(kind='bar')
plt.title('Mushroom Class Distribution')
plt.show()

sns.countplot(x='gill_size', hue='mushroom_class', data=df)
plt.title('Gill Size vs Mushroom Class')
plt.show()

plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title('Feature Correlation')
plt.show()
```

Step 6: Train-Test Split & Scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

x = df.drop('mushroom_class', axis=1, errors='ignore')
y = df['mushroom_class']
```

```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=43)

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test  = scaler.fit_transform(x_test)
```

Step 7: PCA Visualization

```
from sklearn.decomposition import PCA

X = df.drop('mushroom_class', axis=1)
y = df['mushroom_class']
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8,6))
for lbl in y.unique():
    plt.scatter(X_pca[y==lbl,0], X_pca[y==lbl,1], label=lbl)
plt.xlabel('PC1'); plt.ylabel('PC2')
plt.title('PCA of Mushroom Dataset')
plt.legend(); plt.show()
print(pca.explained_variance_ratio_)
```

Step 8: ANN — Early Stopping

```
import tensorflow as tf
from tensorflow import keras
from keras import Sequential
from keras.layers import Dense, BatchNormalization, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping

callback = EarlyStopping(
    monitor='val_loss',
    min_delta=0.01,
    patience=5,
    verbose=1,
    mode='auto',
    restore_best_weights=True,
    start_from_epoch=0
)
```

Step 9: ANN — Architecture & Training

```
model = Sequential()
model.add(Dense(32, activation='relu', input_dim=111))
model.add(BatchNormalization())
model.add(Dropout(0.3))
model.add(Dense(16, activation='relu')); model.add(Dropout(0.2))
model.add(Dense(16, activation='relu')); model.add(Dropout(0.2))
model.add(Dense(16, activation='relu')); model.add(Dropout(0.2))
```

```
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer=Adam(learning_rate=0.0001),
              loss='binary_crossentropy', metrics=['accuracy'])
model.summary()

history = model.fit(x_train, y_train, batch_size=32, epochs=300,
                    validation_data=(x_test, y_test), callbacks=[callback])
y_pred_ann = model.predict(x_test).flatten()

print('Train Accuracy:',      history.history['accuracy'][-1])
print('Validation Accuracy:', history.history['val_accuracy'][-1])
```

Step 10: Visualize & Evaluate

```
# Loss & Accuracy curves
plt.plot(history.history['loss'], label='train_loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.legend(); plt.title('Loss Curve'); plt.show()

plt.plot(history.history['accuracy'], label='train_acc')
plt.plot(history.history['val_accuracy'], label='val_acc')
plt.legend(); plt.title('Accuracy Curve'); plt.show()

# Final evaluation
eval_result = model.evaluate(x_test, y_test, batch_size=32, verbose=1)
print('Test Loss:',     eval_result[0])
print('Test Accuracy:', eval_result[1])

# Classification metrics
from sklearn.metrics import (mean_squared_error, mean_absolute_error,
    r2_score, precision_score, recall_score, f1_score, confusion_matrix)
y_pred_prob = model.predict(x_test, batch_size=32)
y_pred = (y_pred_prob > 0.5).astype(int)

print('MSE:', mean_squared_error(y_test, y_pred))
print('MAE:', mean_absolute_error(y_test, y_pred))
print('Precision:', precision_score(y_test, y_pred))
print('Recall:',    recall_score(y_test, y_pred))
print('F1 Score:',  f1_score(y_test, y_pred))
print('R2 Score:',  r2_score(y_test, y_pred))

# Confusion matrix heatmap
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Pred 0','Pred 1'],
            yticklabels=['Actual 0','Actual 1'])
plt.ylabel('Actual'); plt.xlabel('Predicted')
plt.title('Confusion Matrix Heatmap'); plt.show()

# Precision-Recall curve
from sklearn.metrics import precision_recall_curve, auc
p_vals, r_vals, _ = precision_recall_curve(y_test, y_pred_prob)
pr_auc = auc(r_vals, p_vals)
```

```
plt.plot(r_vals, p_vals, lw=2, label=f'AUC = {pr_auc:.3f}')
plt.xlabel('Recall'); plt.ylabel('Precision')
plt.title('Precision-Recall Curve'); plt.legend(); plt.grid(True); plt.show()
```

RESULT

The Artificial Neural Network using Backpropagation was successfully built and trained on the Mushroom dataset. The model achieved high classification accuracy with low binary cross-entropy loss. Early stopping prevented overfitting by monitoring validation loss and restoring the best weights. Metrics including Precision, Recall, F1 Score, and the Confusion Matrix confirmed the model's strong performance in classifying mushrooms as edible or poisonous.

EXPERIMENT 3

K-Nearest Neighbour (KNN) Classification

AIM

1. To implement the K-Nearest Neighbour (KNN) algorithm for classifying the Mushroom dataset.
2. To print both correct and wrong predictions with their actual and predicted labels.
3. To evaluate the model using accuracy score and classification report.

ALGORITHM

14. Load and preprocess the Mushroom dataset (encoding, scaling).
15. Split data into training (75%) and testing (25%) sets.
16. Initialize KNeighborsClassifier with K=5 inside a StandardScaler Pipeline.
17. Fit the pipeline on training data.
18. Predict labels for train and test sets.
19. Calculate and print Train Accuracy and Test Accuracy.
20. Print full Classification Report (Precision, Recall, F1, Support).
21. Display correct and wrong predictions separately.

SOURCE CODE

Step 1: Load Libraries & Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data_set = 'mushroom.csv'
df = pd.read_csv(data_set)
```

Step 2: Data Exploration

```
df.shape
df.head()
df.tail()
df.dtypes
df.info()
df.describe(include='all').T
print(len(df.columns))
```

Step 3: Pre-Processing

3.1 Rename Columns

```
df.columns = [
    'Index','Mushroom_Class','Cap_Shape','Cap_Surface','Cap_Color',
    'Has_Bruises','Odor','Gill_Attachment','Gill_Spacing','Gill_Size',
    'Gill_Color','Stalk_Shape','Stalk_Root','Stalk_Surface_Above_Ring',
```

```
    'Stalk_Surface_Below_Ring','Stalk_Color_Above_Ring','Stalk_Color_Below_Ring',
    'Veil_Type','Veil_Color','Ring_Count','Ring_Type','Spore_Print_Color',
    'Population_Type','Habitat_Type'
]
df.columns = df.columns.str.lower().str.replace(' ', '_')
df = df.drop(columns=['index'], errors='ignore')
```

3.2 Remove Duplicates

```
df.duplicated().sum()       # check
df = df.drop_duplicates()   # remove
df.duplicated().sum()       # verify
```

3.3 Handle Null Values

```
null = df.isnull().sum()
null[null>0].sort_values(ascending=False)

def fill_null_values(df):
    df = df.copy()
    for col in df.select_dtypes(include='object').columns:
        if not df[col].mode().empty:
            df[col] = df[col].fillna(df[col].mode()[0])
    for col in df.select_dtypes(include=['int64','float64']).columns:
        df[col] = df[col].fillna(df[col].mean())
    return df

df = fill_null_values(df)
print('NULL VALUES ARE FILLED')
```

Step 4: Encoding

4.1 Binary Encoding

```
binary_map = {
    'gill_attachment': {'FREE':0, 'ATTACHED':1},
    'gill_spacing':    {'CROWDED':0, 'CLOSE':1},
    'gill_size':       {'NARROW':0, 'BROAD':1},
    'stalk_shape':     {'TAPERING':0, 'ENLARGING':1},
    'ring_count':      {'ONE':1, 'TWO':2, 'NONE':0},
    'veil_type': 1,
}

def apply_binary_encoding(df, binary_maps):
    df = df.copy()
    for col, mapping in binary_maps.items():
        if col in df.columns:
            if isinstance(mapping, dict):
                lower_mapping = {k.lower(): v for k, v in mapping.items()}
                df[col] =
df[col].astype(str).str.strip().str.lower().map(lower_mapping)
            else:
                df[col] = mapping
    return df

df = apply_binary_encoding(df, binary_map)
```

```
print('Binary encoding applied successfully')
```

4.2 One-Hot Encoding

```
from sklearn.preprocessing import OneHotEncoder

one_col = ['cap_shape','cap_surface','cap_color','has_bruises','odor',
           'gill_color','stalk_root','stalk_surface_above_ring',
           'stalk_surface_below_ring','stalk_color_above_ring',
           'stalk_color_below_ring','veil_color','ring_type',
           'spore_print_color','population_type','habitat_type']

encoder = OneHotEncoder(sparse_output=False)
for col in one_col:
    encoded = encoder.fit_transform(df[[col]])
    encoded_df = pd.DataFrame(encoded,
                 columns=encoder.get_feature_names_out([col]),
                 index=df.index)
    df = pd.concat([df.drop(col, axis=1), encoded_df], axis=1)
```

4.3 Label Encoding (Target Column)

```
from sklearn.preprocessing import LabelEncoder

label = LabelEncoder()
df['mushroom_class'] = label.fit_transform(df['mushroom_class'])
df['mushroom_class'].unique()
```

Step 5: EDA

```
df['mushroom_class'].value_counts().plot(kind='bar')
plt.title('Mushroom Class Distribution')
plt.show()

sns.countplot(x='gill_size', hue='mushroom_class', data=df)
plt.title('Gill Size vs Mushroom Class')
plt.show()

plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title('Feature Correlation')
plt.show()
```

Step 6: Train-Test Split & Scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

x = df.drop('mushroom_class', axis=1, errors='ignore')
y = df['mushroom_class']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=43)
```

```
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test  = scaler.fit_transform(x_test)
```

KNN Model — Build & Train

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report

knn_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=5))
])

knn_pipeline.fit(x_train, y_train)
```

KNN Model — Predictions & Evaluation

```
y_train_pred = knn_pipeline.predict(x_train)
y_test_pred  = knn_pipeline.predict(x_test)

print('Train Accuracy:', accuracy_score(y_train, y_train_pred))
print('Test Accuracy:',  accuracy_score(y_test, y_test_pred))

print('\nClassification Report:')
print(classification_report(y_test, y_test_pred))
```

Print Correct & Wrong Predictions

```
import numpy as np

correct = np.where(y_test_pred == y_test)[0]
wrong   = np.where(y_test_pred != y_test)[0]

print(f'\nCorrect Predictions: {len(correct)}')
for i in correct[:10]:   # show first 10
    print(f'  Index {i}: Actual={y_test.iloc[i]}, Predicted={y_test_pred[i]}')

print(f'\nWrong Predictions: {len(wrong)}')
for i in wrong[:10]:     # show first 10
    print(f'  Index {i}: Actual={y_test.iloc[i]}, Predicted={y_test_pred[i]}')
```

RESULT

The K-Nearest Neighbour algorithm was successfully implemented on the Mushroom dataset. With K=5 and a StandardScaler pipeline, the model achieved high train and test accuracy. Both correct and wrong predictions were printed with their actual and predicted labels, and the classification report confirmed strong Precision, Recall, and F1 Score values.

EXPERIMENT 4

K-Means Clustering

AIM

1. To apply the K-Means algorithm to cluster the Mushroom dataset stored in a CSV file.
2. To determine the optimal number of clusters using the Elbow Method.
3. To visualize the clusters using PCA and comment on the quality of clustering.

ALGORITHM

22. Load and preprocess the Mushroom dataset (encoding, scaling).
23. Standardize features using StandardScaler.
24. Apply the Elbow Method: compute inertia for K = 1 to 10 and plot to find the optimal K.
25. Fit KMeans with optimal K (K=2) on the scaled feature matrix.
26. Assign cluster labels to each data point.
27. Reduce dimensions using PCA (2 components) and scatter-plot clusters.
28. Compare clusters with actual class labels using a crosstab to assess quality.

SOURCE CODE

Step 1: Load Libraries & Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data_set = 'mushroom.csv'
df = pd.read_csv(data_set)
```

Step 2: Data Exploration

```
df.shape
df.head()
df.tail()
df.dtypes
df.info()
df.describe(include='all').T
print(len(df.columns))
```

Step 3: Pre-Processing

3.1 Rename Columns

```
df.columns = [
    'Index','Mushroom_Class','Cap_Shape','Cap_Surface','Cap_Color',
    'Has_Bruises','Odor','Gill_Attachment','Gill_Spacing','Gill_Size',
    'Gill_Color','Stalk_Shape','Stalk_Root','Stalk_Surface_Above_Ring',
    'Stalk_Surface_Below_Ring','Stalk_Color_Above_Ring','Stalk_Color_Below_Ring',
```

```
    'Veil_Type','Veil_Color','Ring_Count','Ring_Type','Spore_Print_Color',
    'Population_Type','Habitat_Type'
]
df.columns = df.columns.str.lower().str.replace(' ', '_')
df = df.drop(columns=['index'], errors='ignore')
```

3.2 Remove Duplicates

```
df.duplicated().sum()       # check
df = df.drop_duplicates()   # remove
df.duplicated().sum()       # verify
```

3.3 Handle Null Values

```
null = df.isnull().sum()
null[null>0].sort_values(ascending=False)

def fill_null_values(df):
    df = df.copy()
    for col in df.select_dtypes(include='object').columns:
        if not df[col].mode().empty:
            df[col] = df[col].fillna(df[col].mode()[0])
    for col in df.select_dtypes(include=['int64','float64']).columns:
        df[col] = df[col].fillna(df[col].mean())
    return df

df = fill_null_values(df)
print('NULL VALUES ARE FILLED')
```

Step 4: Encoding

4.1 Binary Encoding

```
binary_map = {
    'gill_attachment': {'FREE':0, 'ATTACHED':1},
    'gill_spacing':    {'CROWDED':0, 'CLOSE':1},
    'gill_size':       {'NARROW':0, 'BROAD':1},
    'stalk_shape':     {'TAPERING':0, 'ENLARGING':1},
    'ring_count':      {'ONE':1, 'TWO':2, 'NONE':0},
    'veil_type': 1,
}

def apply_binary_encoding(df, binary_maps):
    df = df.copy()
    for col, mapping in binary_maps.items():
        if col in df.columns:
            if isinstance(mapping, dict):
                lower_mapping = {k.lower(): v for k, v in mapping.items()}
                df[col] =
df[col].astype(str).str.strip().str.lower().map(lower_mapping)
            else:
                df[col] = mapping
    return df

df = apply_binary_encoding(df, binary_map)
print('Binary encoding applied successfully')
```

4.2 One-Hot Encoding

```
from sklearn.preprocessing import OneHotEncoder

one_col = ['cap_shape','cap_surface','cap_color','has_bruises','odor',
           'gill_color','stalk_root','stalk_surface_above_ring',
           'stalk_surface_below_ring','stalk_color_above_ring',
           'stalk_color_below_ring','veil_color','ring_type',
           'spore_print_color','population_type','habitat_type']

encoder = OneHotEncoder(sparse_output=False)
for col in one_col:
    encoded = encoder.fit_transform(df[[col]])
    encoded_df = pd.DataFrame(encoded,
                 columns=encoder.get_feature_names_out([col]),
                 index=df.index)
    df = pd.concat([df.drop(col, axis=1), encoded_df], axis=1)
```

4.3 Label Encoding (Target Column)

```
from sklearn.preprocessing import LabelEncoder

label = LabelEncoder()
df['mushroom_class'] = label.fit_transform(df['mushroom_class'])
df['mushroom_class'].unique()
```

Step 5: EDA

```
df['mushroom_class'].value_counts().plot(kind='bar')
plt.title('Mushroom Class Distribution')
plt.show()

sns.countplot(x='gill_size', hue='mushroom_class', data=df)
plt.title('Gill Size vs Mushroom Class')
plt.show()

plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title('Feature Correlation')
plt.show()
```

Step 6: Train-Test Split & Scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

x = df.drop('mushroom_class', axis=1, errors='ignore')
y = df['mushroom_class']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=43)

scaler = StandardScaler()
```

```
x_train = scaler.fit_transform(x_train)
x_test  = scaler.fit_transform(x_test)
```

Standardize Features for Clustering

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X = df.drop('mushroom_class', axis=1)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Elbow Method

```
from sklearn.cluster import KMeans

inertia = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

plt.plot(range(1, 11), inertia, marker='o')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()
```

Fit K-Means (K=2) & Visualize

```
kmeans = KMeans(n_clusters=2, random_state=42)
clusters = kmeans.fit_predict(X_scaled)

df['Cluster'] = clusters

# PCA for 2D visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap='viridis')
plt.xlabel('PC1'); plt.ylabel('PC2')
plt.title('K-Means Clustering (PCA View)')
plt.colorbar(label='Cluster')
plt.show()
```

Quality Assessment — Cluster vs. Actual Class

```
# Cross-tabulation: Cluster vs actual mushroom_class
crosstab = pd.crosstab(df['Cluster'], df['mushroom_class'])
print(crosstab)

# Silhouette Score (clustering quality metric, 1 = perfect)
from sklearn.metrics import silhouette_score
```

```
score = silhouette_score(X_scaled, clusters)
print(f'Silhouette Score: {score:.4f}')
```

RESULT

The K-Means clustering algorithm was successfully applied to the Mushroom dataset. The Elbow Method indicated K=2 as the optimal number of clusters, which aligns with the binary nature of the dataset (edible vs. poisonous). PCA scatter plots confirmed well-separated clusters, and the crosstab with actual labels showed strong correspondence between cluster assignments and true classes. The Silhouette Score confirmed the quality of clustering.

EXPERIMENT 5

Principal Component Analysis (PCA)

AIM

1. To interpret the results of PCA analysis applied to the Mushroom dataset.
2. To reduce high-dimensional feature space to 2 principal components for visualization.
3. To compute and interpret the explained variance ratio and cumulative variance.

ALGORITHM

29. Load and preprocess the Mushroom dataset (encoding, scaling).
30. Standardize all features using StandardScaler (zero mean, unit variance).
31. Apply PCA with n_components=2 to reduce dimensions.
32. Plot the 2D scatter of principal components colored by mushroom class.
33. Print the explained_variance_ratio_ to interpret how much variance each component captures.
34. Plot cumulative explained variance to decide the ideal number of components.
35. Interpret the loading/contribution of original features on each principal component.

SOURCE CODE

Step 1: Load Libraries & Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data_set = 'mushroom.csv'
df = pd.read_csv(data_set)
```

Step 2: Data Exploration

```
df.shape
df.head()
df.tail()
df.dtypes
df.info()
df.describe(include='all').T
print(len(df.columns))
```

Step 3: Pre-Processing

3.1 Rename Columns

```
df.columns = [
    'Index','Mushroom_Class','Cap_Shape','Cap_Surface','Cap_Color',
    'Has_Bruises','Odor','Gill_Attachment','Gill_Spacing','Gill_Size',
    'Gill_Color','Stalk_Shape','Stalk_Root','Stalk_Surface_Above_Ring',
    'Stalk_Surface_Below_Ring','Stalk_Color_Above_Ring','Stalk_Color_Below_Ring',
```

```
    'Veil_Type','Veil_Color','Ring_Count','Ring_Type','Spore_Print_Color',
    'Population_Type','Habitat_Type'
]
df.columns = df.columns.str.lower().str.replace(' ', '_')
df = df.drop(columns=['index'], errors='ignore')
```

3.2 Remove Duplicates

```
df.duplicated().sum()       # check
df = df.drop_duplicates()   # remove
df.duplicated().sum()       # verify
```

3.3 Handle Null Values

```
null = df.isnull().sum()
null[null>0].sort_values(ascending=False)

def fill_null_values(df):
    df = df.copy()
    for col in df.select_dtypes(include='object').columns:
        if not df[col].mode().empty:
            df[col] = df[col].fillna(df[col].mode()[0])
    for col in df.select_dtypes(include=['int64','float64']).columns:
        df[col] = df[col].fillna(df[col].mean())
    return df

df = fill_null_values(df)
print('NULL VALUES ARE FILLED')
```

Step 4: Encoding

4.1 Binary Encoding

```
binary_map = {
    'gill_attachment': {'FREE':0, 'ATTACHED':1},
    'gill_spacing':    {'CROWDED':0, 'CLOSE':1},
    'gill_size':       {'NARROW':0, 'BROAD':1},
    'stalk_shape':     {'TAPERING':0, 'ENLARGING':1},
    'ring_count':      {'ONE':1, 'TWO':2, 'NONE':0},
    'veil_type': 1,
}

def apply_binary_encoding(df, binary_maps):
    df = df.copy()
    for col, mapping in binary_maps.items():
        if col in df.columns:
            if isinstance(mapping, dict):
                lower_mapping = {k.lower(): v for k, v in mapping.items()}
                df[col] =
df[col].astype(str).str.strip().str.lower().map(lower_mapping)
            else:
                df[col] = mapping
    return df

df = apply_binary_encoding(df, binary_map)
print('Binary encoding applied successfully')
```

4.2 One-Hot Encoding

```
from sklearn.preprocessing import OneHotEncoder

one_col = ['cap_shape','cap_surface','cap_color','has_bruises','odor',
           'gill_color','stalk_root','stalk_surface_above_ring',
           'stalk_surface_below_ring','stalk_color_above_ring',
           'stalk_color_below_ring','veil_color','ring_type',
           'spore_print_color','population_type','habitat_type']

encoder = OneHotEncoder(sparse_output=False)
for col in one_col:
    encoded = encoder.fit_transform(df[[col]])
    encoded_df = pd.DataFrame(encoded,
                 columns=encoder.get_feature_names_out([col]),
                 index=df.index)
    df = pd.concat([df.drop(col, axis=1), encoded_df], axis=1)
```

4.3 Label Encoding (Target Column)

```
from sklearn.preprocessing import LabelEncoder

label = LabelEncoder()
df['mushroom_class'] = label.fit_transform(df['mushroom_class'])
df['mushroom_class'].unique()
```

Step 5: EDA

```
df['mushroom_class'].value_counts().plot(kind='bar')
plt.title('Mushroom Class Distribution')
plt.show()

sns.countplot(x='gill_size', hue='mushroom_class', data=df)
plt.title('Gill Size vs Mushroom Class')
plt.show()

plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title('Feature Correlation')
plt.show()
```

Step 6: Train-Test Split & Scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

x = df.drop('mushroom_class', axis=1, errors='ignore')
y = df['mushroom_class']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=43)

scaler = StandardScaler()
```

```
x_train = scaler.fit_transform(x_train)
x_test  = scaler.fit_transform(x_test)
```

Standardize & Apply PCA

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X = df.drop('mushroom_class', axis=1)
y = df['mushroom_class']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

2D Scatter — Visualize Classes in PCA Space

```
plt.figure(figsize=(8, 6))
for lbl in y.unique():
    plt.scatter(X_pca[y == lbl, 0], X_pca[y == lbl, 1],
                label=f'Class {lbl}', alpha=0.6)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA of Mushroom Dataset')
plt.legend(); plt.show()
```

Explained Variance Analysis

```
print('Explained Variance Ratio:', pca.explained_variance_ratio_)
print('Total Variance Captured:', pca.explained_variance_ratio_.sum())

# Cumulative variance with all components
pca_full = PCA().fit(X_scaled)
cumvar = pca_full.explained_variance_ratio_.cumsum()

plt.figure(figsize=(9, 5))
plt.plot(cumvar, marker='o')
plt.axhline(y=0.95, color='r', linestyle='--', label='95% threshold')
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Cumulative Explained Variance vs. No. of Components')
plt.legend(); plt.grid(True); plt.show()

n_95 = (cumvar >= 0.95).argmax() + 1
print(f'Components needed for 95% variance: {n_95}')
```

Feature Loadings (Component Contributions)

```
# Feature names after encoding
feature_names = df.drop('mushroom_class', axis=1).columns
```

```
loadings = pd.DataFrame(
    pca.components_.T,
    index=feature_names,
    columns=['PC1', 'PC2']
)

print('Top 10 Features Contributing to PC1:')
print(loadings['PC1'].abs().sort_values(ascending=False).head(10))

print('\nTop 10 Features Contributing to PC2:')
print(loadings['PC2'].abs().sort_values(ascending=False).head(10))
```

RESULT

PCA was successfully applied to the Mushroom dataset. The first two principal components captured a significant portion of the total variance. The 2D scatter plot showed clear class separation between edible and poisonous mushrooms in the reduced PCA space. The cumulative variance plot was used to determine the minimum number of components required to retain 95% of the variance. Feature loadings revealed which original mushroom attributes contributed most to each principal component.

EXPERIMENT 6

Naive Bayesian Classifier

AIM

1. To implement the Naive Bayesian classifier on the Mushroom dataset stored as a CSV file.
2. To compute the accuracy of the classifier considering training and test datasets.
3. To validate the model using 5-fold cross-validation and print a classification report.

ALGORITHM

36. Load and preprocess the Mushroom dataset (encoding, scaling).
37. Split data into training (80%) and testing (20%) sets with stratified sampling.
38. Standardize features using StandardScaler (GaussianNB works best with normalized features).
39. Instantiate GaussianNB classifier and fit on training data.
40. Predict on train and test sets.
41. Compute and print Train Accuracy, Test Accuracy.
42. Print full Classification Report (Precision, Recall, F1, Support).
43. Perform 5-fold Cross-Validation and report mean accuracy.

SOURCE CODE

Step 1: Load Libraries & Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data_set = 'mushroom.csv'
df = pd.read_csv(data_set)
```

Step 2: Data Exploration

```
df.shape
df.head()
df.tail()
df.dtypes
df.info()
df.describe(include='all').T
print(len(df.columns))
```

Step 3: Pre-Processing

3.1 Rename Columns

```
df.columns = [
    'Index','Mushroom_Class','Cap_Shape','Cap_Surface','Cap_Color',
    'Has_Bruises','Odor','Gill_Attachment','Gill_Spacing','Gill_Size',
    'Gill_Color','Stalk_Shape','Stalk_Root','Stalk_Surface_Above_Ring',
```

```
    'Stalk_Surface_Below_Ring','Stalk_Color_Above_Ring','Stalk_Color_Below_Ring',
    'Veil_Type','Veil_Color','Ring_Count','Ring_Type','Spore_Print_Color',
    'Population_Type','Habitat_Type'
]
df.columns = df.columns.str.lower().str.replace(' ', '_')
df = df.drop(columns=['index'], errors='ignore')
```

3.2 Remove Duplicates

```
df.duplicated().sum()       # check
df = df.drop_duplicates()   # remove
df.duplicated().sum()       # verify
```

3.3 Handle Null Values

```
null = df.isnull().sum()
null[null>0].sort_values(ascending=False)

def fill_null_values(df):
    df = df.copy()
    for col in df.select_dtypes(include='object').columns:
        if not df[col].mode().empty:
            df[col] = df[col].fillna(df[col].mode()[0])
    for col in df.select_dtypes(include=['int64','float64']).columns:
        df[col] = df[col].fillna(df[col].mean())
    return df

df = fill_null_values(df)
print('NULL VALUES ARE FILLED')
```

Step 4: Encoding

4.1 Binary Encoding

```
binary_map = {
    'gill_attachment': {'FREE':0, 'ATTACHED':1},
    'gill_spacing':    {'CROWDED':0, 'CLOSE':1},
    'gill_size':       {'NARROW':0, 'BROAD':1},
    'stalk_shape':     {'TAPERING':0, 'ENLARGING':1},
    'ring_count':      {'ONE':1, 'TWO':2, 'NONE':0},
    'veil_type': 1,
}

def apply_binary_encoding(df, binary_maps):
    df = df.copy()
    for col, mapping in binary_maps.items():
        if col in df.columns:
            if isinstance(mapping, dict):
                lower_mapping = {k.lower(): v for k, v in mapping.items()}
                df[col] =
df[col].astype(str).str.strip().str.lower().map(lower_mapping)
            else:
                df[col] = mapping
    return df

df = apply_binary_encoding(df, binary_map)
```

```
print('Binary encoding applied successfully')
```

4.2 One-Hot Encoding

```
from sklearn.preprocessing import OneHotEncoder

one_col = ['cap_shape','cap_surface','cap_color','has_bruises','odor',
           'gill_color','stalk_root','stalk_surface_above_ring',
           'stalk_surface_below_ring','stalk_color_above_ring',
           'stalk_color_below_ring','veil_color','ring_type',
           'spore_print_color','population_type','habitat_type']

encoder = OneHotEncoder(sparse_output=False)
for col in one_col:
    encoded = encoder.fit_transform(df[[col]])
    encoded_df = pd.DataFrame(encoded,
                 columns=encoder.get_feature_names_out([col]),
                 index=df.index)
    df = pd.concat([df.drop(col, axis=1), encoded_df], axis=1)
```

4.3 Label Encoding (Target Column)

```
from sklearn.preprocessing import LabelEncoder

label = LabelEncoder()
df['mushroom_class'] = label.fit_transform(df['mushroom_class'])
df['mushroom_class'].unique()
```

Step 5: EDA

```
df['mushroom_class'].value_counts().plot(kind='bar')
plt.title('Mushroom Class Distribution')
plt.show()

sns.countplot(x='gill_size', hue='mushroom_class', data=df)
plt.title('Gill Size vs Mushroom Class')
plt.show()

plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title('Feature Correlation')
plt.show()
```

Step 6: Train-Test Split & Scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

x = df.drop('mushroom_class', axis=1, errors='ignore')
y = df['mushroom_class']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=43)
```

```
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test  = scaler.fit_transform(x_test)
```

Stratified Train-Test Split (80/20)

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

X = df.drop('mushroom_class', axis=1)
y = df['mushroom_class']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)
```

Naive Bayes — Fit & Predict

```
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)

y_train_pred = nb_model.predict(X_train)
y_test_pred  = nb_model.predict(X_test)

print('Train Accuracy:', accuracy_score(y_train, y_train_pred))
print('Test Accuracy:',  accuracy_score(y_test, y_test_pred))
```

Classification Report

```
print('\nClassification Report:')
print(classification_report(y_test, y_test_pred))
```

5-Fold Cross-Validation

```
# Re-fit on original unscaled data for cross_val_score
X_raw = df.drop('mushroom_class', axis=1)
y_raw = df['mushroom_class']

scores = cross_val_score(GaussianNB(), X_raw, y_raw, cv=5)

print('CV Fold Scores:', scores)
print('Mean CV Accuracy:', scores.mean())
print('Std Dev:', scores.std())

# Class distribution
print('\nClass Distribution:')
print(y_raw.value_counts())
```

Confusion Matrix Visualization

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_test_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',
            xticklabels=['Pred Edible','Pred Poisonous'],
            yticklabels=['Actual Edible','Actual Poisonous'])
plt.ylabel('Actual'); plt.xlabel('Predicted')
plt.title('Naive Bayes — Confusion Matrix'); plt.show()
```

RESULT

The Naive Bayesian classifier (GaussianNB) was successfully implemented on the Mushroom dataset. The classifier achieved high accuracy on both training and test sets. The 5-fold cross-validation confirmed consistent model performance with minimal variance across folds. The classification report showed strong Precision, Recall, and F1 Score for both classes (edible and poisonous mushrooms), validating the effectiveness of the Naive Bayes model.